

****1. AI Studio Setup****

``` The webapp is called SiteSync. Initialize a modern frontend project using Next.js alongside Tailwind CSS. Recreate the dark, technical aesthetic from the reference image by utilizing a deep black background, dark gray panels, and neon yellow accents for active states, borders, and progress indicators. Implement a monospace typography for IDs, status badges, and technical labels, while using a clean sans-serif font for standard paragraph text.```

## **\*\*2. Firebase Setup\*\***

``` Develop the core application shell featuring a fixed navigation sidebar on the left. Integrate the required menu items consisting of Dashboard, Properties, Contacts, Deals, Calendar, and Emails, placing the user profile badge in the bottom left corner. Construct a top header containing a global search input, a notification icon, and a theme toggle switch, ensuring the dark mode is set as the strict default state.```

``` Connect to Firebase using this configuration: [your config]

Develop dedicated login and registration screens that strictly adhere to the established SiteSync design system. Implement the email and password authentication methods to allow seamless account creation and login for the live demonstration, ensuring session persistence so the user is securely redirected to the dashboard after authenticating.```

## **\*\*3. Firestore & Mapbox\*\***

``` We need to completely overhaul the current dashboard because it currently looks like a server monitoring tool instead of a real estate CRM. Please replace the Total Uptime and Active Nodes widgets with CRM-specific metrics like Total Active Deals, New Leads, and Monthly Revenue. Swap out the Global Traffic Map with the Mapbox 3D integration we planned, centering it on our active property portfolio. Finally, change the System Logs terminal into a live Activity Stream that fetches recent user actions and CRM events directly from our Firestore database.```

__Datastore Rules__

``` rules\_version = '2';

```

service cloud.firestore {
 match /databases/{database}/documents {
 match /{document=**}{
 allow read, write: if request.auth != null;
 }
 }
}

```

#### **\*\*4. API & Debugging\*\***

```` The mapbox integration on the front dashboard isn't working properly. Also the user email is shown wrong in the bottom left. ````

****5. Firebase Storage****

```` Develop the main properties interface to display a highly visual grid of all real estate assets fetched directly from the database. Create a detailed view for individual properties that includes a dedicated image gallery. Integrate Firebase Storage by building a drag-and-drop upload zone where users can securely upload high-resolution property images. Ensure the upload function retrieves the final download URLs from Storage and saves them directly into an array within the specific property's Firestore document. ````

#### **\*\*6. Property Panel\*\***

\_\_Storage Rules\_\_

```` rules\_version = '2';

```

service firebase.storage {
  match /b/{bucket}/o {
    match /{allPaths=**}{
      allow read, write: if request.auth != null;
    }
  }
}

```

```
}  
  
}  
  
} `` `
```

****7. Refining Properties****

`` ` Activate the "Add Property" button in the top right of the current Properties view. Clicking it should trigger a modern slide-over panel sliding in from the right, perfectly matching the dark SiteSync theme with its neon yellow accents. Since brokers and housing companies need to capture a massive amount of data, strictly divide this form into logical tabs or a wizard flow to keep the UI clean and prevent user fatigue. In the first tab, capture the basic data like the internal title, marketing type (rent or sale), and the exact property type. The second tab handles the geographical address data. The third tab is reserved for the hard facts: living space, lot size, number of rooms, bathrooms, year of construction, as well as exact pricing details including additional costs and potential commission. Also, make sure to integrate fields for legal requirements like the energy certificate value and heating type here.

Once the property is created and the user navigates to the detailed property view, implement the panel for matching prospects right below the core property information. Develop a real-time query function that searches the contacts database in Firestore. The system must cross-reference the parameters of the currently displayed property, specifically the price and room count, with the saved search profile fields of the clients. Render the exact matches as compact, clickable profile cards in this section. This setup allows the broker to instantly see which client meets the criteria and, with a single click directly from the property view, open a new deal in the pipeline or draft a direct message.`` `

`` ` Please implement the following critical updates and bug fixes across the application in a single sprint. First, replace the text-based logo with a modern, abstract neon green icon to avoid negative associations and improve branding. In the property slide-over panel, drastically increase the visibility of the "Save Property" button by giving it a solid neon green background with black text so it acts as a clear primary call-to-action. Next, expand the property management capabilities by adding "Edit" and "Delete" buttons to the property detail view. The edit function must reopen the slide-over panel pre-filled with the current

Firestore data for updating, while the delete function must trigger a strict confirmation modal before permanently removing the document from Firestore and its associated images from Firebase Storage. Address the Next.js console warnings by adding a sizes property to the Image components that use fill (e.g., sizes="(max-width: 768px) 100vw, 50vw") to optimize loading. Resolve the chart rendering error by assigning an explicit minHeight to the parent container of the Recharts component so it does not render with a negative size. Finally, repair the global light/dark mode toggle to ensure it correctly switches the application theme, and implement a functional popover menu on the bottom-left user badge that includes a logout option to properly terminate the active Firebase session.``

****8. Contacts & Deals****

`` Develop the Contacts management interface to serve as the central hub for all client interactions. Construct a high-performance data table to list all contacts, ensuring the UI strictly follows the established dark SiteSync theme with neon green accents. Implement full CRUD functionality by adding accessible options to create, edit, and safely delete contact profiles, utilizing the slide-over panel pattern for data entry. When defining the contact schema in Firestore, include essential real estate CRM fields: basic personal information, contact categorization (e.g., buyer, tenant, owner, investor), lead status, and specifically their detailed search criteria such as maximum budget, minimum room count, and preferred locations. Establish a robust bidirectional relationship between the contacts and properties collections in Firestore. This architecture must allow users to manually link a contact to specific properties, marking them as actively interested directly from the UI. Finally, leverage the search criteria fields to power the automated matching engine. The system must dynamically cross-reference property attributes against these contact search profiles and automatically render the matching clients as suggested leads in the designated prospects section at the bottom of the corresponding property detail window.``

`` Develop the "Deals" interface as a dynamic, interactive Kanban board to visually manage sales and rental pipelines, strictly adhering to the dark SiteSync UI theme. Since the CRM targets both real estate agents and housing societies, build the board with customizable or dual-track pipeline stages (e.g., Lead, Viewing, Credit Check, Negotiation, Notary/Contract, Closed). Ensure every deal card is relationally linked to specific documents in the contacts and properties Firestore collections, displaying the client's name, the property title, and the financial value (expected commission or rent) directly on

the card UI. Implement smooth drag-and-drop functionality (e.g., using @hello-pangea/dnd) so users can move deals between stages, ensuring every drop triggers an immediate backend update in Firestore to persistently save the new deal status. When a deal card is clicked, open a detailed slide-over panel that includes a dedicated document management section linked to Firebase Storage—crucial for securely handling lease agreements, credit checks, and notary drafts—alongside a chronological activity feed to log the deal's progression. `` `

****9. Action Buttons****

`` ` Implement the functionality for the quick action buttons on the property detail view. For the "Generate Brochure" button, integrate a robust client-side PDF generation library such as react-pdf or jsPDF. This function must dynamically pull the specific property details, the high-resolution images stored in Firebase Storage, and the new SiteSync logo to compile a professionally formatted, downloadable brochure. For the "Schedule Viewing" button, create a modal that prompts the user to select an interested contact from the database and then seamlessly opens a scheduling interface to pick an available date and time, saving this new appointment directly into the Firestore database. `` `

****10. Refinement****

`` ` Fix the global search bar in the top header so it actively queries the Firestore database and returns relevant results. Upgrade the Mapbox integration on the dashboard to dynamically fetch all properties from Firestore. Use the property addresses to place markers on the map, automatically calculating the bounding box (fitBounds) so all properties are visible on load. Apply distinct color-coding to the markers based on property status or type. Enable the 3D building layer. Finally, add interactive popups to the markers: clicking a marker must reveal a small card with the property image, key metrics, and a direct link to the property detail page. `` `

`` ` Update the styling of the "Create Deal" button in the pipeline view, giving it a high-contrast neon green background to ensure it is clearly visible. Make the stages of deals colored so its clearly visible what deal is in which current state of the deal. Make sure to add a connection to the property and contact that are linked in each deal so you can click on the deal and jump straight to the contact or property when clicking on it. The searchbar at the top should give all the results separated by category. It should give contacts,

properties, deals or even calendar entries. You can also search for property facts like number of bathrooms or the street or city. `` `

****11. Interactive Map****

`` ` Relocate the Mapbox 3D component from the Dashboard to the Properties module. Reconfigure the Properties page layout to feature a split-view or a prominent toggle function (Grid/List vs. Map View). The map must act as a global portfolio overview, dynamically rendering markers for every property document retrieved from the Firestore properties collection based on its address coordinates. Ensure that clicking a map marker triggers a compact info-window displaying the property's thumbnail, title, and key metrics, along with a functional link to navigate to the full property detail view. `` `

`` ` The Map view in the property panel isn't working. Fix this and make sure to show all the properties listed in the firestore database on the map. Use the address information from the properties to find the building. Mark the whole 3d Building or plot (when there is no 3D Building). `` `

****12. Security****

`` ` Create a simple landing password page before the signup/signin. The password should be [your password] and it should grant access to the login/signup when input is correct. `` `